Test Suite

In simple term:

- It's a collection of related test cases grouped together so that they can be executed as a unit.
- We can create multiple test suites based on our need.

Real world use cases:

1. PR (Pull Request) validation:
   Instead of running all test cases for every PR, run only P0 (critical) Test Suite.

2. Hotfix or targeted change:

Example: A bank flow hotfix, we can only run Bank test suite (this saves time and reduces unnecessary test execution)

3. Better maintainability and organization:

   Grouping of test cases feature wise:

   For example:

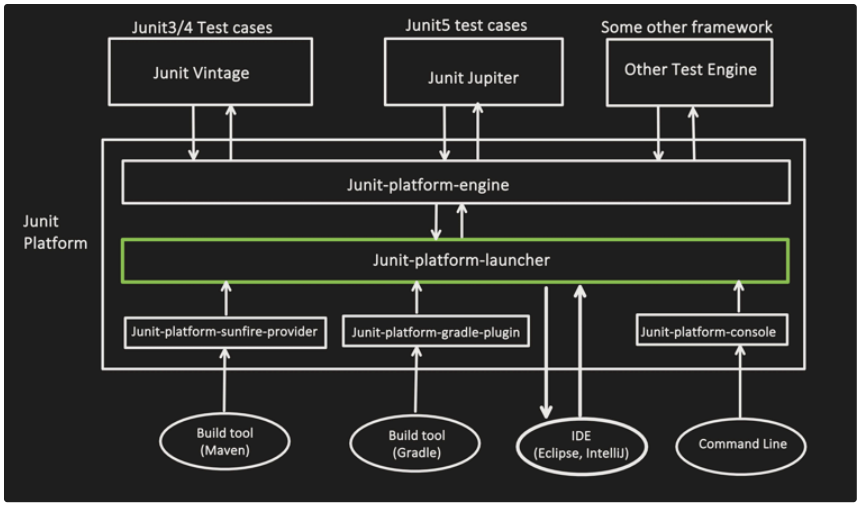
Concept && Coding

- Instrument wise grouping:
  - Bank Test Suite
  - Card Test Suite
  - Balance Test Suite etc.

- Country wise grouping:
  - India Test Suite
  - US Test Suite etc.

In little different way, if we try to understand Test suite:

- Test Suite logic is run by "Junit-platform-launcher".



Test suite works in 2 logical phases:

Test Suite does 2 things in order:

Step1: "Discovery":

Identify all the test cases which are candidate to be grouped together.

- Based on Class
- Based on Package

Concept && Coding

Step2: "Filtering":

the test cases which are discovered in step1, run filter on top of it using:

- IncludeTags
- ExcludeTags
- IncludeClassNamePatterns
  etc.

At this step, some discovered tests are discarded.
And the final filtered set is executed.

Example of Test Suite:

1. **Dependency**

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>
</parent>

```

this internally pulls:

- junit-jupiter-api
- junit-jupiter-engine
- junit-platform-launcher

And we know that, Test Suite logic is run by junit-platform-launcher, but it can not understand Test Suite specific annotations.

For that, we need to add junit-platform-suite dependency.

```
<dependency>
    <groupId>org.junit.platform</groupId>
    <artifactId>junit-platform-suite</artifactId>
    <version>1.10.2</version>
    <scope>test</scope>
</dependency>

```

2. **Discovery Step**

Test Suite need **selectors**, these selectors decides where to look for test cases.

```
import org.junit.platform.suite.api.Suite;

/*
This annotation tells Junit that, this is a Test suite class,
use this to discover the test cases which need to be grouped together.

*/
@Suite
public class MySuite {
}

```

Concept && Coding

But here we have not added any selector, so it will not be able to discover any test cases.

```
Tests failed: 1 of 1 test
/Users/shrayanshjain/Library/Java/JavaVirtualMachines/ms-21.0.8/Contents/Home/bin/java ...

org.junit.platform.suite.engine.NoTestsDiscoveredException: Suite [concepts.TestSuite.MySuite] did not discover any tests
<2 internal lines>
    at java.base/java.util.Iterator.forEachRemaining(Iterator.java:133) <7 internal lines>


Process finished with exit code 255
```

Lets add Selector, we have different selector:

- @SelectPackages
- @SelectClasses
- @SelectMethod and @SelectMethods

1. **@SelectPackages**

Include or Search all Test classes and Test method within this package and its subpackages.

```
import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;

```

```java
@Suite
  /*
  These are packages, Test Suite will discover all the test c
  lasses and test methods present inside these 3 packages and
  their subpackages too (recursively)
  */
@SelectPackages({
        "concepts.TaggingAndFilter",
        "concepts.RepeatedTest",
        "concepts.ParameterisedTest"
})
public class MySuite {
}

```

Test run output:

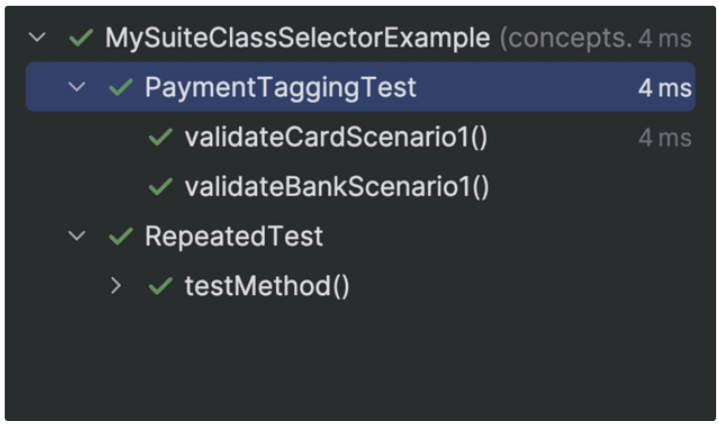
Concept && Coding

2. **@SelectClasses**

Include or Search specific Test classes and Test method within those classes only.

```java
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({
        concepts.TaggingAndFilter.PaymentTaggingTest.class,
        concepts.RepeatedTest.RepeatedTest.class
})
public class MySuiteClassSelectorExample {
}

```

Test run output:



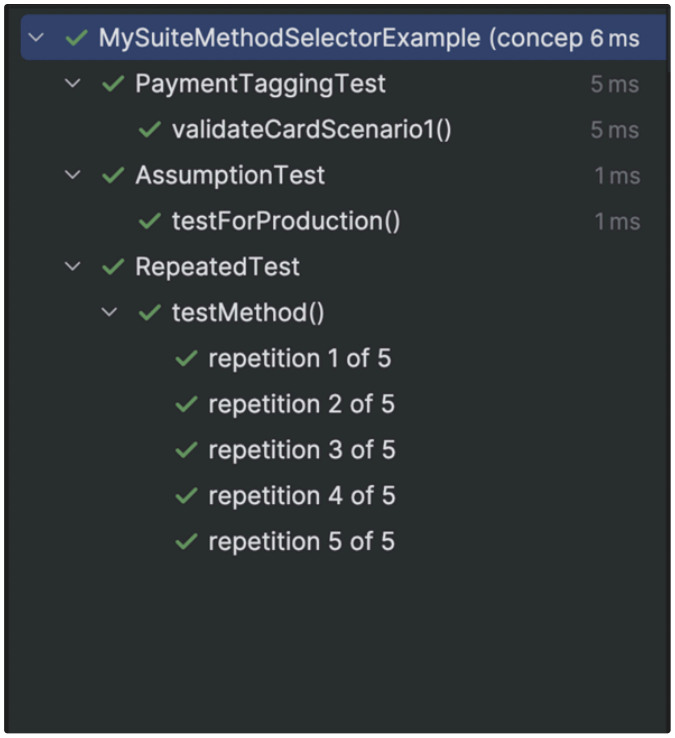
3. **@SelectMethod and @SelectMethods**

- Include or Search specific Test method only.
- Works reliably only for non parametrized test methods.

```java
import org.junit.platform.suite.api.SelectMethod;
import org.junit.platform.suite.api.SelectMethods;
import org.junit.platform.suite.api.Suite;

@Suite
/* for selecting single test method */
@SelectMethod(
        type = concepts.TaggingAndFilter.PaymentTaggingTest.class,
        name = "validateCardScenario1"
)

/* for selecting multiple test methods */
@SelectMethods({
        @SelectMethod(
                type = concepts.Assumptions.AssumptionTest.class,
                name = "testForProduction"
        ),
        @SelectMethod(
                type = concepts.RepeatedTest.RepeatedTest.class,
                name = "testMethod"
        )
})
public class MySuiteMethodSelectorExample {
}

```

Concept && Coding

Test run output:



If we try to run for parametrized test method, it wont be able to resolve it.

This is the Test Method which I want to add in Test Suite:

Concept && Coding

```java
public class MyStringUtilTest {

    /******************* Single parameter *******************/
    /*
     @ValueSource
     */

    @ParameterizedTest
    @ValueSource(strings = {"civic", "level", "madam"})
    public void testPalindrome(String word) {
        assertTrue(MyStringUtil.isPalindrome(word));
    }
}

```

```java
import org.junit.platform.suite.api.SelectMethod;
import org.junit.platform.suite.api.Suite;

@Suite
/* for selecting single test method */
@SelectMethod(
        type = concepts.ParameterisedTest.MyStringUtilTest.class,
        name = "testPalindrome"
)
public class MySuiteMethodSelectorExample {
}

```

Internally it will try to look for method "testPalindrome()", but there is no such method in MyStringUtilTest class.

```
/Users/shrayanshjain/Library/Java/JavaVirtualMachines/ms-21.0.8/Contents/Home/bin/java ...
Internal Error occurred.
org.junit.platform.commons.JUnitException Create breakpoint : TestEngine with ID 'junit-platform-suite' failed to discover tests <18 internal lines>
Caused by: org.junit.platform.commons.JUnitException Create breakpoint : TestEngine with ID 'junit-jupiter' failed to discover tests <8 internal lines>
    at java.base/java.util.Iterator.forEachRemaining(Iterator.java:133) <11 internal lines>
    ... 17 more
Caused by: org.junit.platform.commons.JUnitException Create breakpoint : MethodSelector [className = 'concepts.ParameterisedTest.MyStringUtilTest', methodN
    ... 36 more
```

3. **Filtering Step**

Filters decide *which* tests run from the already discovered test set. *Below are frequently used filters (there are more apart from these)*

- @IncludePackages
- @ExcludePackages
- @IncludeTags
- @ExcludeTags
- @IncludeClassNamePatterns
- @ExcludeClassNamePatterns
- @IncludeEngines
- @ExcludeEngines

@IncludePackages:

```java
@Suite
@SelectPackages({
        "concepts"
})
@IncludePackages({
        "concepts.TaggingAndFilter",
        "concepts.RepeatedTest"
})
public class MySuiteFilterExample {
}

```

Concept && Coding

Within "concepts" package:

1. *during discovery phase it selected multiple packages:*

    concepts.Assumptions

    concepts.ParameterisedTest

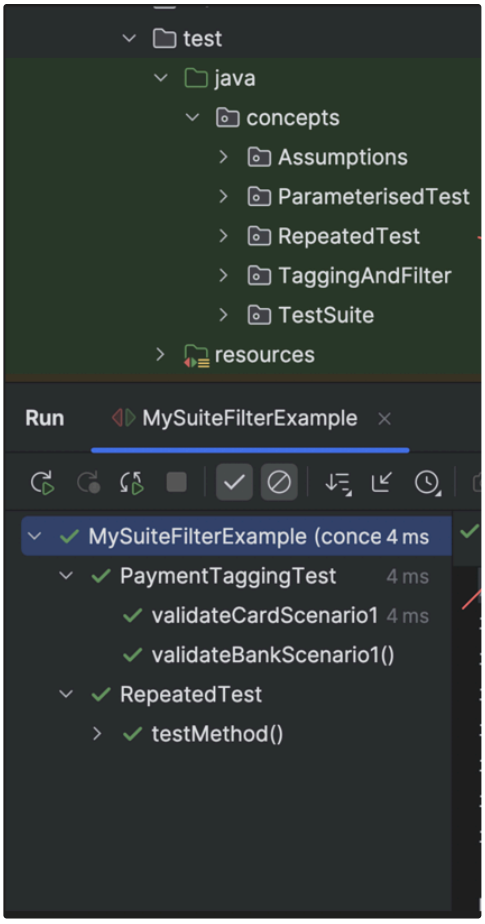
    concepts.RepeatedTest

    concepts.TaggingAndFilter

2. *@IncludePackages: we filter out the packages which we actually want and only these packages are included.*

Output:

Out of multiple packages, TaggingAndFilter and RepeatedTest package is included, rest all packages are excluded



Concept && Coding

@ExcludePackages

```
@Suite
@SelectPackages({
        "concepts"
})
@ExcludePackages({
        "concepts.ParameterisedTest",
        "concepts.TestSuite"
})
public class MySuiteFilterExample {
}

```

Within "concepts" package:

1. *during discovery phase it selected multiple packages:*

   concepts.Assumptions

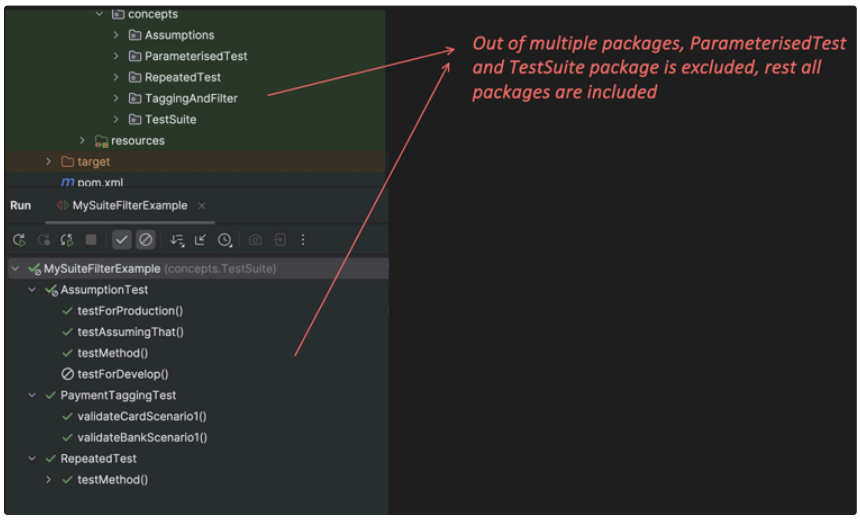
   concepts.ParameterisedTest

   concepts.RepeatedTest

   concepts.TaggingAndFilter

2. *@ExcludePackages: we filter out the packages which we don't want, and rest all packages are included.*

Output:



When both used together:

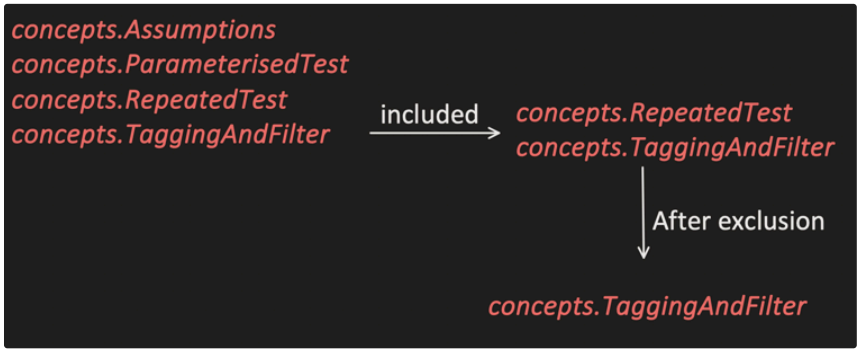
```
@Suite
@SelectPackages({
        "concepts"
})
@IncludePackages({
        "concepts.TaggingAndFilter",
        "concepts.RepeatedTest"
})
@ExcludePackages({
        "concepts.ParameterisedTest",
        "concepts.TestSuite",
        "concepts.RepeatedTest"
})
public class MySuiteFilterExample {
}

```

Concept && Coding

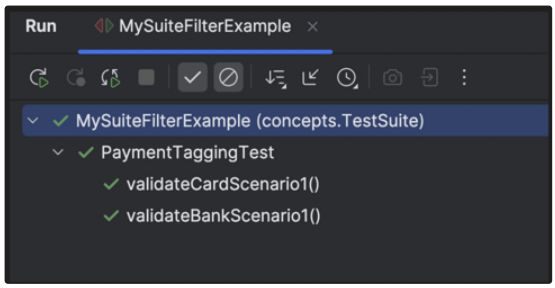
Suite first do:

(Discovery ∩ inclusion Filters) - Exclusion Filters

∩ - *intersection, common elements between 2 sets*



Output:



@IncludeTags

Run only those test method that has one of the given tags.

```
@Suite
@SelectPackages({
        "concepts"
})
@IncludeTags({
        "card",
        "bank"
})
public class MySuiteFilterExample {
}

```

This concepts package (and its subpackages) might have 1000s of test cases, but IncludeTags filter only include those test cases which has one of these tags:

@Tag("card")

or

@Tag("bank")

Concept && Coding

@ExcludeTags

Remove those test method that has one of the given tags. All other test methods are included.

```
@Suite
@SelectPackages({
        "concepts"
})
@ExcludeTags({
        "bank"
})
public class MySuiteFilterExample {
}

```

@IncludeClassNamePatterns and @ExcludeClassNamePatterns

IncludeClassNamePatterns : Runs test classes whose name matches regex pattern

```
@IncludeClassNamePatterns({
        ".*Test"
})
public class MySuiteFilterExample {
```

```
}

```

ExcludeClassNamePatterns : skip test classes whose name matches regex pattern and include others

```
@ExcludeClassNamePatterns({
        ".*Example"
})
public class MySuiteFilterExample {
}

```

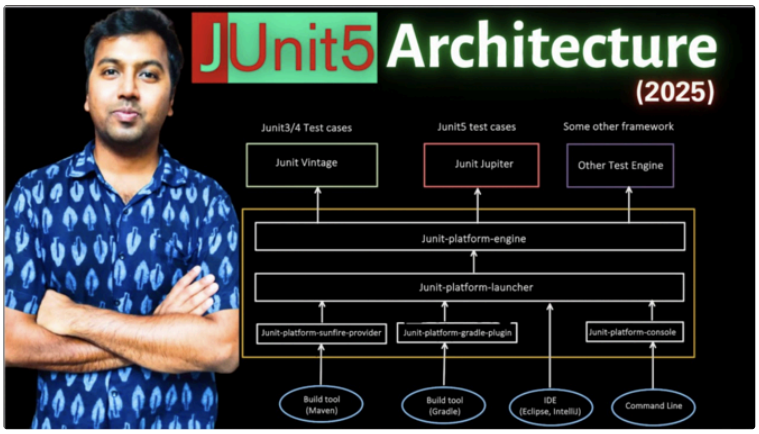
'.'	Any single character	s.j Matches: s_j , anything 1 character between s and j
'*'	Zero or more times	sj* Matches: sja Sjaa
'.*'	anything	.*payment.* anything before + payment + anything after
'^'	Start of text	^Payment PaymentTest - Match CardPaymentTest - does not match
'|'	OR	.*(payment|order).* Matches either the substring payment or substring order anywhere in the text. (payment|order)$ Only matches if it ends with payment or order.
(?i)	Case insensitive	By default regex is case sensitive .*payment cardPaymentTest - do not match (?i).*payment

Concept && Coding

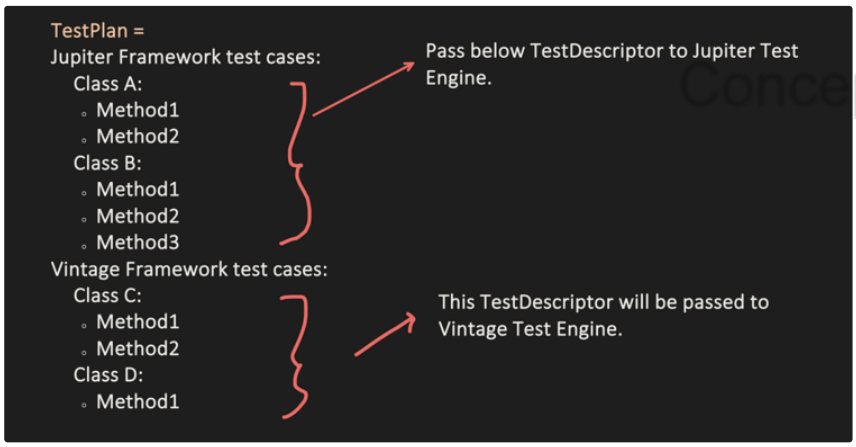
		we are telling that anything after (?i) should be case insensitive.

@IncludeEngines and **@ExcludeEngines**

If we recall from the Architecture video:



Platform initiate Discovery Phase → Each engine returns the test which belongs to them → Platform creates Test plan



Concept && Coding

So platform has information about which test case belongs to which engine.

```
@IncludeEngines({
        "junit-jupiter"
})
@ExcludeEngines({
        "junit-vintage"
})
public class MySuiteFilterExample {
}

```

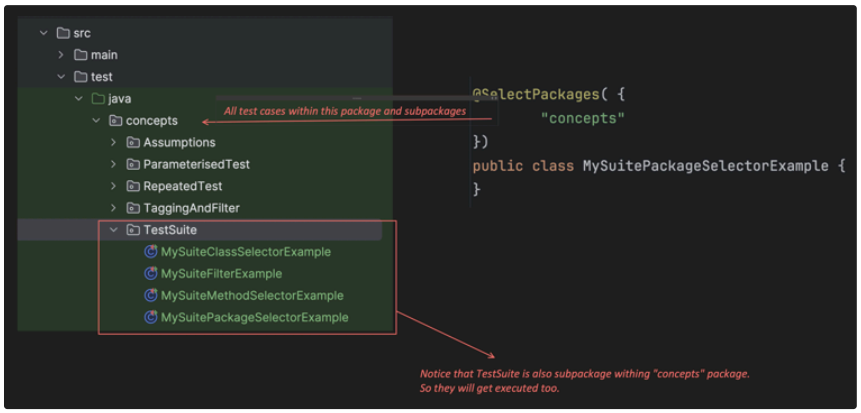
We can run the Suite using maven command:

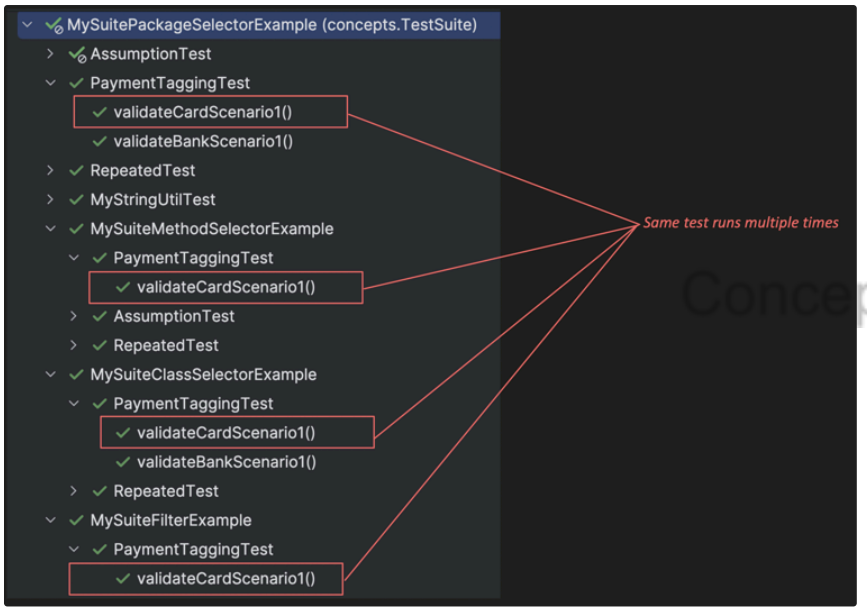
Same we can configure in CI job:

mvn test -Dtest=MySuite1,MySuite2

Before we wrap up: 2 important points

1st: Avoid duplicate test run





Concept && Coding

2nd: Difference between Tagging with Filter and Test Suite

Both are industry standard, its not like one wins over another.

Tagging with Filter:

- Code:

```
@Tag("slow")
class PaymentTaggingTest {

@Test
@Tag("card")
@Tag("visa")
Void validateCardScenario1() {
     System.out.println("inside card test case");
```

```
}

@Test
@Tag("bank")
Void validateBankScenario1() {
     System.out.println("inside bank test case");
 }

}

```

To execute:

mvn test -PtagTest -Dgroups="bank | card"

- Dynamically (during runtime), we are deciding, which tests to run.
- Because of Dynamic nature, more preferred in CI jobs as it provides flexibility (no code change required).
- Less risk of duplicate test case run
- Low maintenance, no extra java classes. Only annotation on existing test classes and methods.

Test Suite:

Concept && Coding

- Code

```
@Suite
@SelectPackages({
        "concepts.Bank",
        "concepts.Card"
})
public class MySuitePackageSelectorExample {
}

```

- A proper Java class, which hardcode(static) which test to discover and need to be grouped together.
- It is more readable. A suite class clearly shows what belongs to a group.
- Chance of duplicate test case run
- Little extra overhead to maintain test suite classes, any change in grouping logic, need to change in those classes too.